

Subida de archivos

Programación Orientada a Objetos – ISAM 2025

¿Cómo trabaja Laravel con los archivos?

- Al ejecutar `php artisan serve` el servidor de Laravel ejecuta la aplicación leyendo solamente el directorio `public`.
- Laravel permite trabajar con un directorio de archivos local o con un servicio de archivos de internet (por ejemplo, Amazon S3). Cada una de estas opciones se conocen como “discos”.
- Si elegimos el disco local, Laravel guarda por defecto todos los archivos subidos en la carpeta `storage` de nuestra aplicación. Esta carpeta no se sube al repositorio.
- Para que `storage` pueda ser leída desde `public` debemos crear un **enlace simbólico** o “*symlink*”.

Crear el formulario

- Para que un formulario pueda recibir archivos para subida, debe declarar obligatoriamente el atributo `enctype="multipart/form-data"`.
- También debe declarar la directiva `@csrf`.

Controlador de archivos

El controlador debe realizar las siguientes tareas:

1. Validar el tipo y tamaño de los archivos subidos.
2. Validar si el usuario tiene permiso de subir archivos.
3. Mover el archivo subido en el formulario al disco local (u otro disco definido).
4. Guardar el nombre del archivo en la base de datos (si fuera necesario).
5. Devolver los mensajes de error/éxito de la operación.

Creación del symlink

- Ingresamos por consola el siguiente comando:

```
php artisan storage:link
```

- Esto creará un symlink en la carpeta public que apuntará a la carpeta Storage.

Mostrar un archivo en mi aplicación

- El trabajo con archivos se realiza mediante la clase Storage.
- Para mostrar un archivo:

```
$archivo = Storage::get('archivo.jpg');
```

- Para descargar un archivo:

```
return Storage::download('archivo.jpg');
```

- Para obtener la URL de un archivo:

```
$url = Storage::url('archivo.jpg');
```

Validar datos

- **Opcion 1:** usar el método validate() sobre el objeto \$request

```
$request->validate([  
    'email' => ['required', 'email'],  
    'mensaje' => ['required', 'min:10', 'max:250']  
]);
```

Opción 2: Usar un FormRequest

- Crear el Form Request desde la consola

```
php artisan make:request ContactoGuardarRequest
```

- Modificar el método authorize()

```
return true;
```

- Trasladar las reglas de validación previas al método rules()
- Modificar el método del controlador para agregar el Form Request en lugar del Request tradicional.

```
use App\Http\Requests\ContactoGuardarRequest;
```

```
public function guardar(ContactoGuardarRequest $request)
```


Extra: redirigir a la página anterior con éxito

- Si la validación es exitosa podemos optar por redirigir al usuario a la página previa usando el método ***redirect()*** o sus variantes, junto con el método ***with()*** que permite enviar datos específicos.

```
// Controlador
```

```
return back()->with('status', 'ok');
```

```
// Vista
```

```
@if(session('status')):
```

```
    <div class="my-3 bg-green-200 text-green-900 rounded p-3">
```

```
        <p>El mensaje ha sido enviado con éxito</p>
```

```
    </div>
```

```
@endif
```